ORGNZ-03: Bucket Strategy and Design

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 3 of 10 | **Created:** January 2026 | **Last Updated:** 05/06/2026

Overview

A well-defined bucket strategy optimizes query performance, controls costs, and ensures compliance. This notebook covers naming conventions, retention planning, organizational alignment, and common bucket design patterns.

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	`storage:bucket-definitions:read`, `storage:logs:read`
Knowledge	Completed ORGNZ-02 (Understanding Grail Buckets)
Data	At least 1 hour of log data

Table of Contents

Learning Objectives

By the end of this notebook, you will:
- Design effective bucket naming conventions
- Plan retention strategies for different use cases
- Understand cost implications of bucket design
- Apply organizational alignment patterns

Bucket Naming Conventions

Recommended Naming Patterns

Pattern	Format	Example	Use Case
Provider-based	`<provider>_<type>_<retention>`	`aws_logs_35d`	Multi-cloud environments
LOB-based	`<provider>_<type>_<lob>`	`aws_logs_finance`	Line of business cost attribution
Team-based	`team_<name>_<type>`	`team_platform_logs`	Team ownership and billing
Compliance	`<regulation>_<type>_<retention>`	`hipaa_logs_7y`	Regulatory requirements

Detailed Examples

Bucket Name	Provider	Data Type	Retention	Org Unit
`aws_logs_35d_platform`	AWS	logs	35 days	Platform team
`azure_metrics_90d_finance`	Azure	metrics	90 days	Finance
`gcp_spans_14d_checkout`	GCP	spans	14 days	Checkout service

Bucket Name	Provider	Data Type	Retention	Org Unit
`audit_logs_365d_compliance`	Any	logs	365 days	Compliance
`debug_logs_7d`	Any	logs	7 days	Development

Naming Rules Reminder

Rule	Requirement
First character	Must be a letter
Allowed characters	Lowercase alphanumeric, underscores, hyphens
Case	Lowercase only
Reserved names	Cannot use `default_*` prefix
Immutability	Names cannot be changed after creation

Retention Strategy

Retention Period Guidelines

Use Case	Recommended Retention	Rationale
Debug/Development	3-7 days	Short-term troubleshooting, high volume
Standard Operations	35 days	Monthly trends, incident response
Performance Analysis	60-90 days	Quarterly comparisons
Compliance/Audit	365-3657 days	Regulatory requirements

Retention by Data Type

Data Type	Typical Retention	Notes
Debug logs	3-7 days	High volume, low long-term value
Application logs	35-90 days	Operational analysis

Data Type	Typical Retention	Notes
Audit logs	1-7 years	Compliance requirements
Metrics	35-90 days	Trending and alerting
Spans	10-35 days	APM troubleshooting
Security events	90-365 days	Security investigations

Bucket Design Patterns

Pattern 1: Long-Term Compliance Data

Store critical audit information for extended periods:

```
Bucket: compliance_audit_logs
Table: logs
Retention: 1825 days (5 years)
Use case: SOX compliance, security audits
Route via: OpenPipeline filter on audit-level logs
```

Pattern 2: Short-Term Debug Logs

Reduce costs by storing verbose logs briefly:

```
Bucket: debug_logs_7d
Table: logs
Retention: 7 days
Use case: Development troubleshooting
Route via: OpenPipeline filter on loglevel=DEBUG
```

Pattern 3: Team Isolation

Separate buckets for team-based access and cost attribution:

```
Bucket: team_platform_logs
Table: logs
Retention: 35 days
Use case: Platform team owns infrastructure logs
Access: IAM policy restricts to platform team
```

Pattern 4: Application-Specific Retention

Business units with specific retention needs:

```
Bucket: finance_app_logs
Table: logs
Retention: 180 days
Use case: Financial application compliance
Cost: Chargeback to Finance cost center
```

Cost Control and Attribution

Cost Factors

Factor	Impact on Cost
Ingest volume (GiB/day)	Direct relationship
Retention period	Storage costs increase with longer retention
Query frequency	Query costs accumulate

Cost Attribution Strategy

Organization Level	Bucket Strategy
Business Unit	One bucket per BU
Cost Center	Map buckets to cost centers
Cloud Account	Separate by AWS/Azure/GCP account
Application	Critical apps get dedicated buckets

Analyzing Bucket Usage

Sprint 1.337 (April 2026) Updates

New default bucket: `default_database_monitoring`. Logs from official Dynatrace database extensions are now routed by default to a dedicated `default_database_monitoring` bucket — keeping them out of `default_logs` for faster queries and tighter IAM scoping. Note that the `default_*` prefix is reserved (per the Naming Rules above) — only Dynatrace-provided buckets use it; you cannot create your own.

OneAgent primary fields enable richer routing. OneAgent now enriches all telemetry at the source with standardized **primary fields** from the Semantic Dictionary (e.g., `dt.security_context`, `dt.cost.costcenter`, `dt.cost.product`) plus customer-defined **primary tags**. These appear as top-level fields on metrics, spans, logs, business events, and Smartscape entities (HOST, PROCESS, CONTAINER, DISK, NETWORK_INTERFACE).

In bucket-routing terms, this means OpenPipeline `route` rules can dispatch on:

```
processors:
  - type: route
    rules:
      - condition: "dt.cost.costcenter == 'cc-1234'"
        destination: "finance_logs"
      - condition: "dt.security_context contains 'pci'"
        destination: "pci_audit_logs_365d"
      - condition: "dt.cost.product == 'checkout'"
        destination: "checkout_logs"
    default: "default_logs"
```

Available only on Latest Dynatrace tenants. See **ORGNZ-06: Security Context** for primary-tag schema design and **OPLOGS** / **OPIPE** for the ingest-time enrichment configuration.

Smartscape Ownership integration. Smartscape entities now carry ownership information that Dynatrace Workflows can read directly — useful when bucket-routing decisions depend on which team owns the producing host or service. See **WFLOW** for the routing-on-ownership pattern.

Bucket Strategy Considerations

Access Control Options

Buckets can serve as an access control mechanism:

Access Method	Description	When to Use
Security context (record-level)	Fine-grained ABAC at the record level	Complex multi-team scenarios
Bucket-level IAM policies	Restrict access to entire buckets	Simple team isolation

Access Method	Description	When to Use
Combined approach	Security context + bucket policies	Maximum control and flexibility

Bucket-Based Team Isolation Example:

```
Team A: Access to team_a_logs bucket only
Team B: Access to team_b_logs bucket only
Platform: Access to all buckets
```

Query Performance at Scale

Bucket sizing directly impacts queryability due to the 500 GB scan limit:

Bucket Ingest	Impact
~1 TB/day	Optimal - can query ~12 hours of data
1-2 TB/day	Acceptable — reduced query window
>2 TB/day	Split — Dynatrace recommends splitting above this threshold

Data Immutability

Operation	Possible?	Alternative
Move data between buckets	No	Route new data via OpenPipeline
Selective record deletion	No	Wait for retention to expire
Bucket deletion	Yes	Deletes ALL data permanently

Routing Data to Buckets

Use OpenPipeline to route data to custom buckets:

```
# OpenPipeline routing rule example
processors:
  - type: route
    rules:
      - condition: "loglevel == 'DEBUG'"
        destination: "debug_logs_7d"
      - condition: "log.source contains 'audit'"
        destination: "audit_logs_365d"
      - condition: "host.group starts-with 'finance-'"
        destination: "finance_app_logs"
    default: "default_logs"
```

Bucket Design Checklist

Use this checklist when planning buckets:

- [ ] Defined naming convention aligned with organization
- [ ] Identified retention requirements by data type
- [ ] Mapped organizational units to buckets for cost attribution
- [ ] Calculated expected ingest volumes (keep <1 TB/day)
- [ ] Planned for compliance requirements
- [ ] Documented bucket purposes
- [ ] Planned OpenPipeline routing rules
- [ ] Considered access control strategy (bucket vs security context)

Next Steps

Continue with the ORGNZ series:
- **ORGNZ-04**: Permissions in Grail Overview

References

- Grail Buckets
- Data Retention
- OpenPipeline Routing

This notebook was AI-generated from Dynatrace documentation and enterprise best practices. It is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.

Query Billing Model

Each custom Grail bucket offers **two query billing models** — choose at creation time based on your workload:

Model	How Billed	Best For
Usage-based	Charged per GiB scanned at query time	Buckets queried infrequently or unpredictably
Retain with Included Queries	Flat rate — query costs included in retention price	Buckets queried frequently or heavily by dashboards/alerts

> The billing model cannot be changed after bucket creation. For compliance and audit buckets queried daily by security dashboards, **Retain with Included Queries** typically lowers total cost. For debug or ephemeral buckets queried only during incidents, **Usage-based** avoids paying for query capacity you rarely use.

Excluding Logs from Storage

When you extract metrics from logs via OpenPipeline (e.g., parsing error rates into a metric), you may not need to retain the source log records at all. Configure a **No Storage Assignment** in the OpenPipeline Storage stage to drop matching records after processing — this avoids retaining data purely for a signal you've already transformed.

> Use with caution: once records are dropped they cannot be recovered. Only use No Storage Assignment when metrics extraction fully captures the required signal.

DQL: Capacity Planning

Track ingest trends to validate routing rules and spot anomalies:

```
// Track log ingest trends by bucket — use for capacity planning and routing
anomaly detection
fetch logs, from:-1h
| summarize recordCount = count(), by:{time_bucket = bin(timestamp, 1h),
dt.system.bucket}
| sort time_bucket desc
```